Copilot Workflow Guide: Plan → Prompt → Build

This is the core workflow you'll follow for each build phase. It keeps you in control, breaks work into manageable pieces, and gives you reusable prompts you can refer back to.

The Pattern

1. Plan (Plan Agent)

Switch to **Plan agent** in Copilot and describe what you want to accomplish for this phase.

Example for Phase 1:

> "I want to build an invoice validator. I'll use React for the frontend and Express for the API. Help me plan the file structure, components, API endpoints, and the steps to get a working end-to-end flow."

Example for Phase 2:

> "I have a working invoice validator with a React frontend and Express API. Now I want to add validation rules for required fields, date checks, and amount validation. I also want to improve the results display with pass/fail indicators. Help me plan the steps."

Review the plan. Ask follow-up questions. Adjust until it makes sense.

2. Generate Step-by-Step Prompts (Agent Mode)

Switch to **Agent Mode** and ask Copilot to turn the plan into individual prompts:

> "Take this plan and break it down into a series of focused prompts I can use one at a time. Each prompt should tackle one piece — don't combine multiple steps. Don't execute them yet, just give me the list of prompts."

You should get back something like:

1. *"Set up a new Express project in the api/ folder with a package.json and basic server on port 3001"*
2. *"Create a React component with a form that has fields for vendor name, amount, date, and PO number"*
3. *"Add a POST /validate endpoint that accepts JSON and returns a validation response"*
4. *"Wire the form to call the API on submit and display the result"*

3. Save the Prompts to a Markdown File (Agent Mode)

Important: Still in **Agent Mode**, ask Copilot to save the prompts to a markdown file in your project:

> "Save these prompts as a checklist to a file called phase1-prompts.md in my project root."

Do this for each phase:

`phase1-prompts.md`

`phase2-prompts.md`

This gives you:

A checklist to track progress

A reference you can go back to if you get lost

A record of your approach for the demo presentation

Reusable prompts for future projects

You can check off each prompt as you complete it:

```markdown
# Phase 1 Prompts

[x] Set up Express project in api/ folder
[x] Create React form component with invoice fields
[ ] Add POST /validate endpoint
[ ] Wire form to API and display results
```

4. Execute One Prompt at a Time (Agent Mode)

Stay in **Agent Mode** and feed each prompt to Copilot individually. After each one:

Review what Copilot generated — do you understand it?

Test that it works — run the app, check the output

Check off the prompt in your markdown file

Move to the next prompt

If something doesn't work, paste the error into Copilot Chat and ask for help before moving on.

Why This Pattern Works

You stay in control. You're directing Copilot step by step, not hoping one big prompt works.

Each step is testable. You can verify each piece works before building on it.

You learn as you go. Reviewing each output builds understanding of the code.

You have a record. The prompt markdown files document your journey for the demo.

Repeat for Each Phase

Use this same pattern at the start of each build phase:

1. **Plan** what you want to accomplish in this phase
2. **Generate prompts** from the plan
3. **Save them** to a markdown file
4. **Execute** one at a time

Variation Ideas to Try

If your team finishes early, or wants to compare approaches, run one or two of these experiments and note what changed.

1. Model Comparison (Same Task, Different LLM)

Pick one prompt from your checklist and run it with two different models.

Track:

Code quality/readability

Correctness on first try

Amount of follow-up prompting needed

Speed to a working result

Use this starter prompt:

> "Run this exact task with Model A and Model B. For each, explain tradeoffs in code quality, completeness, and likely bugs."

2. Prompt Detail A/B Test

Try the same implementation step with:

Version A: short prompt

Version B: highly specific prompt (constraints, file names, expected behavior)

Compare which one gives less rework.

3. Role-Based Prompting

Ask Copilot to respond in different roles for the same task:

"Act as a senior backend engineer"

"Act as a test engineer"

"Act as a code reviewer"

See how output style and risk awareness change.

4. Test-First Variation

Before generating implementation code, ask for tests first.

Example:

> "Write Playwright tests for happy path and one validation failure first. Then implement only the code needed to pass them."

This usually improves clarity and reduces hidden regressions.

5. Selector Quality Challenge (Playwright)

Take one generated spec and improve selectors using accessibility-first locators.

Prompt:

> "Refactor this test to prefer getByRole/getByLabel selectors and remove brittle CSS selectors. Keep behavior unchanged."

6. Failure Injection Drill

Intentionally break one assumption and fix it with Copilot:

API returns 500

Network delay/timeout

Missing required field

Goal: practice debugging prompts, not just generation prompts.

7. Refactor Pass (No New Features)

After a phase works, run a cleanup-only pass:

Better naming

Smaller functions/components

Clearer validation messages

Remove dead code

Prompt:

> "Refactor this code for readability and maintainability only. Do not change behavior. Explain each refactor briefly."

8. Explain-Back Check

After Copilot generates code, ask it to explain:

What changed

Why it works

What could still fail

If the explanation is weak, the implementation may also be weak.

9. Timebox Challenge

Set a 10-minute limit for one checklist item:

1. Plan prompt
2. Generate code
3. Run and test
4. Capture what blocked you

This sharpens prompt quality and decision-making under time pressure.

10. Demo Readiness Variant

Have Copilot produce a short demo script from your completed checklist:

> "Create a 2-minute demo walkthrough of what we built, what failed, what we fixed, and what we'd do next."

This turns your markdown prompts into a stronger final presentation.

Recommended Variation Matrix by Phase

Use this to pick the most useful experiments for each phase.

| Phase | Primary Goal | Recommended Variations (Pick 2-3) | Why These Fit |

|---|---|---|---|

| Phase 1: Build a working end-to-end flow | Ship a stable happy path quickly | Test-First Variation, Selector Quality Challenge, Prompt Detail A/B Test | Keeps scope small, establishes good testing habits early, and improves first-pass generation quality |

| Phase 2: Add validation and richer behavior | Improve correctness and edge-case handling | Failure Injection Drill, Explain-Back Check, Role-Based Prompting | Surfaces weak assumptions, strengthens

reasoning, and improves handling of invalid inputs |

| Phase 3: Polish, reliability, and demo readiness | Make the project resilient and presentation-ready | Model Comparison, Refactor Pass, Demo Readiness Variant | Helps choose the best final implementation style, reduces technical debt, and sharpens final storytelling |

Quick Selection Rules

If your app is unstable: do **Selector Quality Challenge** and **Failure Injection Drill** first.

If your prompts feel inconsistent: do **Prompt Detail A/B Test** and **Model Comparison**.

If demo day is close: do **Refactor Pass** and **Demo Readiness Variant**.